

Air University
Department of Computer Science
Introduction to Parallel & Distributed Computing
Lab Manual

Lab 03:
Processes and Threads

Item	Details
Course	Introduction to Parallel & Distributed Computing
Course Code	CS426L
Duration	3 Hours
Instructor	Muhammad Nauman Zafar Hashmi (Lecturer, CS Dept.)
Semester	VII C (FALL26)

1. Lab Overview

Item	Details
Lab Number	03
Lab Title	Processes and Threads
Duration	3 Hours
Programming Language	C# (.NET, System.Diagnostics, System.Threading)
Development Environment	.NET SDK, Visual Studio Code with the C# Dev Kit, Windows terminal / VS Code integrated terminal (same setup as Lab 02; no WSL is required for this lab)
Main Concepts	Process creation and address-space separation, thread creation with System.Threading, process- vs. thread-creation overhead, thread lifecycle states
Main C# Features / Tools	System.Diagnostics.Process, Process.Start(), Process.Id, System.Threading.Thread, Thread.Start(), Thread.Join(), Thread.ThreadState, System.Diagnostics.Stopwatch

After completing this laboratory, students should be able to:

1. use `System.Diagnostics.Process.Start()` to create a child process and demonstrate that the parent and child processes occupy separate, independent address spaces;
2. implement a C# threading program (using `System.Threading`) that sums slices of an array across multiple worker threads and combines their partial results;
3. measure and compare the overhead of creating a process against the overhead of creating a thread on the same machine, using `System.Diagnostics.Stopwatch`;
4. explain, referencing the standard thread lifecycle states (New, Runnable/Ready, Running, Blocked/Waiting, Terminated) and .NET's `Thread.ThreadState`, why threads are better suited to fine-grained parallel work than processes.

Students should be familiar with basic C# syntax, building and running C# programs with the dotnet CLI or VS Code, and the material covered in Lab 01 (sequential/parallel timing, core count) and Lab 02 (System.Threading.Thread creation with Start()/Join(), Environment.ProcessorCount).

2. Setting Up the Development Environment

This laboratory reuses the same environment configured for Lab 02: Visual Studio Code with the C# Dev Kit extension, and the .NET SDK installed natively on Windows. Unlike Lab 02's Task 1, nothing in this lab requires WSL or any Linux-only tool — System.Diagnostics.Process and System.Threading.Thread both work directly on Windows.

2.1 Confirm the Environment

From a Windows terminal (PowerShell or VS Code's integrated terminal), confirm the .NET SDK is still available:

```
dotnet --version
```

If this command is not recognized, revisit Lab 02, Section 2.2 to (re)install the .NET SDK, and Section 2.1 to (re)install VS Code and the C# Dev Kit.

2.2 Create a Working Folder for This Lab

1. In VS Code, open (or create) a folder for this lab, e.g. C:\Users\<you>\PDC_Lab03.
2. Open the VS Code integrated terminal (Terminal > New Terminal).

Each task below creates its own console project inside this folder using dotnet new console -o <ProjectName>.

3. Background: Processes, Threads, and Overhead

Review the following terminology before starting the tasks.

- **Process:** an independently executing program with its own private virtual address space, its own set of OS-managed resources (open file handles, heap, environment variables), and its own process ID (PID). Two processes cannot directly read or write each other's memory.
- **fork() (POSIX):** on Linux/Unix, fork() duplicates the calling process into a parent and a child that initially share identical (but separate, copy-on-write) memory contents. Windows and .NET do not expose fork(); the closest equivalent available in C# is System.Diagnostics.Process.Start(), which launches a new, independent process running a specified executable — conceptually closer to fork() immediately followed by exec(). This lab uses Process.Start() to demonstrate the same underlying idea fork() is used to teach: address-space separation between parent and child.
- **Thread:** an independent unit of execution that runs inside a process and shares that process's address space (heap, static/global fields) with every other thread in the same process. Threads within one process can read and write the same variables directly, which is both what makes them efficient for fine-grained work and what makes synchronization necessary.
- **Creation overhead:** creating a new process typically requires the operating system to allocate a new virtual address space, duplicate or set up page tables, and initialize a new set of OS resources, which is comparatively expensive. Creating a new thread reuses the existing process's address space and OS resources, and only needs a new stack and scheduling context, which is comparatively cheap. Task 3 measures this difference directly on the lab machine.
- **Thread lifecycle:** conceptually, a thread moves through the states New (created but not yet started), Runnable/Ready (eligible to run, waiting for the scheduler), Running (currently executing on a core), Blocked/Waiting (paused, e.g. waiting on I/O, a lock, or Thread.Join()), and Terminated (finished executing). .NET exposes an approximation of this through the System.Threading.Thread.ThreadState property, examined in Task 4.

4. Task 1: Process Creation and Address-Space Separation

Use `System.Diagnostics.Process.Start()` to create a child process and demonstrate that the parent and child processes occupy separate, independent address spaces.

4.1 Approach

A single compiled program will act as both parent and child, distinguished by a command-line argument. When launched with no arguments, it behaves as the parent: it sets a local counter to a starting value, launches a second copy of itself (passing `--child` on the command line) as a genuinely separate OS process, waits for that child process to exit, and then reports its own counter value. When launched with `--child`, the same program behaves as the child: it independently sets its own counter to the same starting value, modifies it differently, prints its own process ID and final counter value, and exits.

Because the parent and child are separate OS processes, each with its own address space, the child's modifications to its counter have no effect on the parent's counter, and vice versa — even though both started from the same source code and the same initial value.

4.2 Required Steps

1. Create a C# console project named `ProcessLab`.

```
dotnet new console -o ProcessLab
cd ProcessLab
```

2. In `Program.cs`, check whether the program was launched with the argument `--child`. If so, run the child branch; otherwise, run the parent branch.
3. In the child branch: print the child's own process ID (`Environment.ProcessId`), set a local counter to 100, add 50 to it, print the resulting value, and exit.
4. In the parent branch: print the parent's own process ID, set a local counter to 100, add 1 to it, then launch the child process using `Process.Start()`, passing `--child` as an argument and the path to the same compiled executable.
5. In the parent branch, wait for the child process to finish using `Process.WaitForExit()` before continuing.
6. After the child has exited, have the parent print its own final counter value, and explicitly state in the output that the parent's and child's counters were modified independently.
7. Run the program and record the process IDs and final counter values reported by the parent and the child.

4.3 Suggested Program Skeleton (`Program.cs`)

```
using System;
using System.Diagnostics;

class Program
{
    static void Main(string[] args)
    {
        if (args.Length > 0 && args[0] == "--child")
        {
            RunAsChild();
        }
        else
        {
            RunAsParent();
        }
    }

    static void RunAsChild()
    {
        Console.WriteLine($"[Child] PID = {Environment.ProcessId}");
    }
}
```

```

    int counter = 100;
    counter += 50;
    Console.WriteLine($"[Child] final counter = {counter}");
}

static void RunAsParent()
{
    Console.WriteLine($"[Parent] PID = {Environment.ProcessId}");
    int counter = 100;
    counter += 1;

    // Your task: build a ProcessStartInfo pointing at this same executable,
    // with ArgumentList containing "--child", then Process.Start() it and
    // WaitForExit() on the returned Process object.

    Console.WriteLine($"[Parent] final counter = {counter}");
    Console.WriteLine("[Parent] Parent and child counters were modified independently (separate
address spaces).");
}
}

```

Hint: `System.Diagnostics.Process.GetCurrentProcess().MainModule.FileName` (or `Environment.ProcessPath` in .NET 6+) gives the path to the currently running executable, which the parent can pass to `ProcessStartInfo.FileName` when launching the child.

4.4 Record Your Results

Role	Process ID (PID)	Final Counter Value
Parent		
Child		

4.5 Discussion Questions

1. Were the parent's and child's process IDs different? What does this confirm about how `Process.Start()` creates the child?
2. Both the parent and the child set a local counter to 100 and then modified it differently. Did either process's final counter value affect the other's? Explain why, in terms of address-space separation.
3. On Linux, `fork()` duplicates the parent's entire memory image at the moment of the call, so the child briefly starts with an exact copy of the parent's variables. In this C# program using `Process.Start()`, does the child ever see the parent's in-progress counter value (i.e. 101, after the parent's increment) at any point? Explain the difference between `fork()`'s copy-on-write duplication and launching a fresh process with `Process.Start()`.

5. Task 2: Summing Array Slices Across Worker Threads

Implement a C# threading program (using `System.Threading`, in place of `pthread`s) that divides a large array into contiguous slices, has one worker thread compute the sum of each slice, and combines the partial sums into a single total.

5.1 Required Steps

1. Create a C# console project named `ArraySumThreads`.

```

dotnet new console -o ArraySumThreads
cd ArraySumThreads

```

2. Allocate an array of at least 10,000,000 integers or doubles and fill it with values (e.g. sequential numbers, or random values with a fixed seed for reproducibility).

3. Decide on a number of worker threads (e.g. `Environment.ProcessorCount`, as in Lab 02), and divide the array into that many contiguous, roughly equal-sized slices.
4. Allocate an array of partial sums with one slot per worker thread.
5. Write a worker method that receives its thread index, computes the start and end bounds of its slice from that index, sums the elements in that slice, and stores the result in its own slot of the partial-sums array (each thread writes only to its own slot, so no lock is needed).
6. Spawn one thread per slice using `new Thread(...).Start()`, and join every thread using `Thread.Join()` before continuing.
7. After all threads have been joined, add up the partial sums to produce the final total.
8. Separately, compute the same total with a single, straightforward sequential loop over the whole array.
9. Compare the threaded total against the sequential total and print whether they match exactly.

5.2 Suggested Program Skeleton (Program.cs)

```
using System;
using System.Threading;

class Program
{
    static long[] data = new long[10_000_000];
    static long[] partialSums;
    static int numWorkers;

    static void SumSlice(object? arg)
    {
        int idx = (int)arg!;
        int sliceSize = data.Length / numWorkers;
        int start = idx * sliceSize;
        int end = (idx == numWorkers - 1) ? data.Length : start + sliceSize;

        long sum = 0;
        for (int i = start; i < end; i++)
        {
            sum += data[i];
        }
        partialSums[idx] = sum;
    }

    static void Main()
    {
        for (int i = 0; i < data.Length; i++) data[i] = i + 1;

        numWorkers = Environment.ProcessorCount;
        partialSums = new long[numWorkers];
        Thread[] threads = new Thread[numWorkers];

        for (int i = 0; i < numWorkers; i++)
        {
            int idx = i;
            threads[i] = new Thread(() => SumSlice(idx));
            threads[i].Start();
        }
        for (int i = 0; i < numWorkers; i++)
        {
            threads[i].Join();
        }

        long threadedTotal = 0;
        foreach (long partial in partialSums) threadedTotal += partial;
    }
}
```

```

    long sequentialTotal = 0;
    foreach (long value in data) sequentialTotal += value;

    Console.WriteLine($"Threaded total: {threadedTotal}");
    Console.WriteLine($"Sequential total: {sequentialTotal}");
    Console.WriteLine($"Match: {threadedTotal == sequentialTotal}");
}
}

```

5.3 Record Your Results

Array Size	Worker Threads Used	Threaded Total	Sequential Total	Match? (Y/N)

5.4 Discussion Questions

1. Why does each worker thread writing only to its own slot in partialSums avoid the need for a lock, whereas having every thread add directly into one shared long total variable would not?
2. If the array length is not evenly divisible by the number of worker threads, how does the skeleton above avoid dropping the last few elements? Identify the specific line responsible.
3. How does this task relate to Task 1: which parts of the ArraySumThreads program are shared between all the worker threads (as would be true within a single process), and how does that sharing differ from the address-space separation you observed between the parent and child processes in Task 1?

6. Task 3: Measuring Process- vs. Thread-Creation Overhead

Measure and compare the overhead of creating a process against the overhead of creating a thread on the same machine, using System.Diagnostics.Stopwatch.

6.1 Required Steps

1. Create a C# console project named CreationOverhead.

```

dotnet new console -o CreationOverhead
cd CreationOverhead

```

2. Reuse (or copy) the ProcessLab executable from Task 1 as a trivial child target: a program that, when run, does nothing but exit immediately.
3. Using a Stopwatch, time how long it takes to create and wait for a fixed number of trivial child processes (e.g. 50 iterations) using Process.Start() followed by Process.WaitForExit(), and compute the average time per process.
4. Using a separate Stopwatch, time how long it takes to create and join the same fixed number of threads (e.g. 50 iterations) using new Thread(...).Start() followed by Thread.Join(), where each thread's work is trivial (e.g. an empty method body), and compute the average time per thread.
5. Print both averages, and compute how many times more expensive process creation was compared to thread creation on this machine.
6. Run the comparison at least twice and record both sets of results.

6.2 Suggested Program Skeleton (Program.cs)

```

using System;
using System.Diagnostics;
using System.Threading;

class Program

```

```

{
    const int Iterations = 50;

    static void Main()
    {
        var processStopwatch = Stopwatch.StartNew();
        for (int i = 0; i < Iterations; i++)
        {
            // Your task: Process.Start() a trivial executable (e.g. the
            // ProcessLab child target from Task 1) and Process.WaitForExit().
        }
        processStopwatch.Stop();

        var threadStopwatch = Stopwatch.StartNew();
        for (int i = 0; i < Iterations; i++)
        {
            Thread t = new Thread(() => { /* trivial work */ });
            t.Start();
            t.Join();
        }
        threadStopwatch.Stop();

        double avgProcessMs = processStopwatch.Elapsed.TotalMilliseconds / Iterations;
        double avgThreadMs = threadStopwatch.Elapsed.TotalMilliseconds / Iterations;

        Console.WriteLine($"Average process creation time: {avgProcessMs:F3} ms");
        Console.WriteLine($"Average thread creation time: {avgThreadMs:F3} ms");
        Console.WriteLine($"Process creation was {(avgProcessMs / avgThreadMs):F1}x more expensive
than thread creation.");
    }
}

```

6.3 Record Your Results

Run	Avg. Process Creation (ms)	Avg. Thread Creation (ms)	Ratio (Process / Thread)
1			
2			

6.4 Discussion Questions

1. Which was more expensive to create, a process or a thread, and by roughly what factor on your machine?
2. Referencing the Background section, explain in your own words why creating a new process (a new address space and OS resource set) costs more than creating a new thread (which reuses the existing process's address space).
3. Would you expect this ratio to grow, shrink, or stay roughly the same if each trivial unit of work were replaced with something more substantial (e.g. summing a large array, as in Task 2)? Justify your answer.

7. Task 4: Thread Lifecycle States

Explain, referencing the standard thread lifecycle states, why threads are better suited to fine-grained parallel work than processes, and observe .NET's own approximation of these states.

7.1 The Standard Thread Lifecycle

State	Meaning
-------	---------

New	The thread object has been created (e.g. <code>new Thread(...)</code>) but <code>Start()</code> has not yet been called; no OS thread exists for it yet.
Runnable / Ready	<code>Start()</code> has been called; the thread is eligible to run and is waiting for the OS scheduler to assign it a core.
Running	The thread is currently executing instructions on a logical core.
Blocked / Waiting	The thread is not currently executing because it is waiting on something else to complete, e.g. an I/O operation, a lock, or another thread via <code>Join()</code> .
Terminated	The thread's method has returned (or it has otherwise ended); it will not run again.

7.2 .NET's Thread.ThreadState

The `System.Threading.Thread` class exposes a `ThreadState` property (a bitwise combination of flags such as `Unstarted`, `Running`, `WaitSleepJoin`, `Stopped`, and `Background`) that approximates the conceptual states above. It is intended mainly for diagnostics rather than precise control, but sampling it at different points is a useful way to see the lifecycle in action.

7.3 Required Steps

1. Create a C# console project named `ThreadLifecycle`.

```
dotnet new console -o ThreadLifecycle
cd ThreadLifecycle
```

2. Create a `Thread` object for a worker method that sleeps briefly (e.g. `Thread.Sleep(200)`) and then returns, but do not call `Start()` yet. Print `thread.ThreadState` immediately after creation (expected: it includes `Unstarted`).
3. Call `Start()` on the thread. Immediately print `thread.ThreadState` again (it should no longer report `Unstarted`).
4. While the worker is sleeping, print `thread.ThreadState` from the main thread at least once more (it should report a waiting-related state, such as `WaitSleepJoin`, while `Thread.Sleep()` is in progress).
5. Call `Join()` to wait for the worker to finish, then print `thread.ThreadState` one final time (it should report `Stopped`).
6. For each printed state, add a one-line comment in your code mapping it to the corresponding conceptual lifecycle state from Section 7.1 (`New`, `Runnable/Ready`, `Running`, `Blocked/Waiting`, or `Terminated`).

7.4 Suggested Program Skeleton (Program.cs)

```
using System;
using System.Threading;

class Program
{
    static void Worker()
    {
        Thread.Sleep(200);
    }

    static void Main()
    {
        Thread t = new Thread(Worker);
        Console.WriteLine($"After creation: {t.ThreadState}"); // conceptually: New

        t.Start();
        Console.WriteLine($"Immediately after Start(): {t.ThreadState}"); // conceptually: Runnable/Ready or
```


Running

```
Thread.Sleep(50); // give the worker a moment to reach Thread.Sleep(200)
Console.WriteLine($"While worker is sleeping: {t.ThreadState}"); // conceptually: Blocked/Waiting

t.Join();
Console.WriteLine($"After Join() completes: {t.ThreadState}"); // conceptually: Terminated
}
}
```

7.5 Record Your Results

Point in Program	Thread.ThreadState Reported	Conceptual Lifecycle State
Immediately after creation, before Start()		New
Immediately after Start()		Runnable / Ready (or Running)
While the worker is inside Thread.Sleep(200)		Blocked / Waiting
After Join() has returned		Terminated

7.6 Discussion Questions

1. Using your Task 3 measurements and the lifecycle states above, explain why moving a thread through New -> Runnable -> Running is cheaper than the equivalent process-creation path observed in Task 3.
2. A program that needs to perform thousands of small, short-lived units of work (fine-grained parallelism) is generally implemented with threads rather than processes. Using your results from Tasks 2, 3, and this task, construct an argument for why: consider creation overhead, address-space sharing (from Task 2), and how quickly a unit of work can move through the lifecycle states.
3. Give one scenario where the address-space isolation demonstrated in Task 1 (processes, not threads) would be preferred despite its higher creation overhead. Justify your answer.

8. Experimental Practice

For meaningful and reproducible results across this laboratory:

1. close other CPU-intensive applications before running Task 3's overhead comparison, since background load affects both measurements;
2. run each task's program at least twice and confirm the results are consistent between runs;
3. use the Release configuration (dotnet run -c Release) rather than Debug when recording Task 3's timing results, since Debug builds can be noticeably slower;
4. keep the raw console output from each task alongside your completed tables, so recorded values can be verified.

9. Expected Findings

By completing this laboratory, students should observe that:

- a parent process launched with Process.Start() and its child process report different process IDs and never observe each other's in-progress variable modifications, confirming separate address spaces;

- a C# threading program can correctly sum an array by dividing it into slices, one per worker thread, and combining independently-computed partial sums, matching a sequential computation of the same total;
- creating a process is substantially more expensive (typically at least one to two orders of magnitude) than creating a thread on the same machine, because process creation requires setting up a new address space and OS resource set, while thread creation reuses the existing process's;
- Thread.ThreadState visibly changes across a thread's lifetime (from Unstarted, through a running/waiting state, to Stopped), corresponding to the conceptual New -> Runnable -> Running -> Blocked -> Terminated lifecycle;
- threads are better suited than processes to fine-grained parallel work because they are cheaper to create and can directly share data through the process's address space, at the cost of requiring explicit synchronization that processes, being isolated, do not need.

10. Troubleshooting Guide

Problem	Likely Cause	Action
Process.Start() throws Win32Exception (file not found)	The FileName passed to ProcessStartInfo does not correctly point at the compiled executable	Use Environment.ProcessPath (.NET 6+) or Process.GetCurrentProcess().MainModule.FileName to obtain the correct path to the currently running executable, and pass it explicitly.
Child process output does not appear in the terminal	ProcessStartInfo.UseShellExecute was left at its default, or output redirection was not configured	For a simple lab exercise, leave UseShellExecute at its default and do not redirect standard output; the child's Console.WriteLine calls will then appear directly in the same terminal window.
Threaded total and sequential total do not match in Task 2	An off-by-one error in slice bounds, or two threads writing to the same partialSums slot	Print each thread's computed start and end bounds and verify they exactly cover the array with no overlaps or gaps; confirm each thread writes only to partialSums[idx] using its own idx.
Task 3's timing results are inconsistent between runs	Other applications competing for CPU time, or a Debug build being used	Close background applications, increase iterations for more stable averages, and run with dotnet run -c Release.
Thread.ThreadState never appears to show Unstarted	The state was printed after Start() was already called	Make sure the first Console.WriteLine of t.ThreadState occurs before t.Start() is invoked.
Program hangs and never exits	Process.WaitForExit() or Thread.Join() was omitted, or the child process/thread never terminates	Confirm every Process.Start() has a matching WaitForExit() and every Thread.Start() has a matching Join().

11. Submission Requirements

Submit one laboratory report containing:

1. source code for the ProcessLab project and the recorded PID/counter results (Task 1);
2. source code for the ArraySumThreads project and the recorded totals/match results (Task 2);
3. source code for the CreationOverhead project and the recorded timing results (Task 3);

4. source code for the ThreadLifecycle project and the recorded ThreadState results (Task 4);
5. brief answers to all discussion questions in Tasks 1 through 4;
6. representative screenshots showing successful build and execution of each of the four programs.

12. Marking Rubric

Component	Marks	Assessment Focus
Task 1: Process creation and address-space separation	25	Correct use of Process.Start()/WaitForExit(), correct PID and counter reporting, and sound discussion answers
Task 2: Array-sum threading program	25	Correct slice division, correct use of Thread.Start()/Join(), matching threaded and sequential totals
Task 3: Process- vs. thread-creation overhead	20	Correct use of Stopwatch, sound comparison, and well-reasoned discussion answers
Task 4: Thread lifecycle states	20	Correct mapping of Thread.ThreadState to conceptual lifecycle states and well-reasoned discussion answers
Code quality and submission organization	10	Readable code, meaningful identifiers, and a complete, well-organized submission
Total	100	

13. Lab Completion Checklist

Before submitting the laboratory report, verify that:

- the .NET SDK and VS Code with the C# Dev Kit are available (carried over from Lab 02; no WSL needed for this lab).
- ProcessLab builds and runs cleanly, and the parent/child PIDs and counters have been recorded.
- ArraySumThreads builds and runs cleanly, and the threaded total matches the sequential total.
- CreationOverhead builds and runs cleanly in Release configuration, and both timing averages have been recorded.
- ThreadLifecycle builds and runs cleanly, and all four ThreadState readings have been recorded and mapped to their conceptual states.
- all discussion questions across Tasks 1-4 are answered.
- required source code and screenshots are included.

14. Conclusion

This laboratory examined the two fundamental units of execution introduced in this course: processes and threads. Using `System.Diagnostics.Process.Start()` in place of `POSIX fork()`, students demonstrated that a parent and child process occupy genuinely separate address spaces, neither able to observe the other's in-progress state. Using `System.Threading.Thread` in place of `pthread`s, students then implemented a multithreaded array-summation program, exploiting the fact that threads within one process share that process's address space directly. Students measured, on their own machine, that process creation carries substantially higher overhead than thread creation, and traced a thread's progression through .NET's `ThreadState` property, connecting it to the conceptual `New` -> `Runnable` -> `Running` -> `Blocked` -> `Terminated` lifecycle. Together, these observations explain why threads, not

processes, are the standard building block for fine-grained parallel work, while processes remain valuable precisely where their higher cost buys genuine isolation. These concepts extend directly into later laboratories involving synchronization primitives and coordination between concurrent threads.